Universidad de Costa Rica

Sede de Occidente · Recinto de Tacares

MO-0005 Análisis de Algoritmos · Grupo 01 · II Ciclo 2026

Prof. Manfred Mejías Acevedo

PUNTUAL

Entregas tardías, satisfacción y fidelización de clientes en Olist: modelo de aprendizaje automático e indicadores analíticos

Entregable 1 — Documental (10 % de la nota final)

Fecha límite: 24 de noviembre de 2026, en Mediación Virtual

Equipo: **Última Milla Labs**

Autores: Kendall Alvarado Artavia ([carné]); Marcos Ferreto Estrada (C5F092); David Moya Correas ([carné]); Sebastián Sánchez Vega (C4J823)

Empresa de estudio: **Olist** (Brasil)

Repositorio del proyecto: mo0005-ecommerce-ml

Índice

1. Introducción y contexto **3**

2. Parte 1 — Las bases del modelo **3**

2.1 Empresa o institución seleccionada 3

2.2 Objetivos estratégicos de la empresa 4

2.3 Problema y necesidad del negocio 4

2.4 Datos fuente 6

2.5 Infraestructura de base de datos 7

2.6 Estructuras de datos creadas 7

2.7 Carga de los datos en las estructuras 8

2.8 Diccionario de datos 8

2.9 Diagrama entidad-relación 12

3. Parte 2 — Flujo y modelos de aprendizaje automático **12**

3.1 Descripción general del flujo en KNIME 12

3.2 Obtención y calidad de los datos 13

3.3 Transformación de los datos 14

3.4 Modelo 1 — No supervisado: segmentación con k-means 15

3.5 Modelo 2 — Supervisado: predicción de la entrega tardía 17

3.6 Validación de ambos modelos 18

3.7 Integraciones con terceros 19

3.8 Evidencia de ejecución del flujo 19

4. Parte 3 — Visualización e indicadores **19**

4.1 Conexión entre KNIME y Power BI 19

4.2 Indicadores (KPI) 19

4.3 Dashboard 20

4.4 Hallazgos principales 21

4.5 Contraste de las hipótesis 22

5. Conclusiones y recomendaciones **23**

6. Limitaciones y trabajo futuro **24**

7. Referencias y anexos **25**

Anexos **25**

Anexo A. Scripts SQL: estructuras y vistas analíticas 25
Anexo B. Consultas de los indicadores 28
Anexo C. Medidas DAX para Power BI 29
Anexo D. Nodos de KNIME por etapa 30
Anexo E. Glosario 31
Anexo F. Enlaces 32

1. Introducción y contexto

Olist es un integrador de marketplaces brasileño que conecta a pequeños comerciantes con compradores de todo el país. El valor de su modelo depende de que cada pedido llegue completo, a tiempo y con una buena experiencia de compra.

Este proyecto analiza, con el conjunto público de Olist (alrededor de 100 000 pedidos de 2016 a 2018), cómo se relacionan las entregas tardías con la satisfacción de los clientes y con su probabilidad de volver a comprar. La exploración sobre la base de datos muestra que el 8.07 % de los pedidos llegó tarde, que su calificación promedio fue de 2.57 frente a 4.30 en los pedidos puntuales y que solo el 3.12 % de los clientes realizó más de un pedido.

Para apoyar a la empresa se construyó, en KNIME, un flujo que integra y depura los datos almacenados en PostgreSQL; un modelo supervisado que predice la entrega tardía con información disponible al comprar; un modelo no supervisado que segmenta los pedidos en perfiles; y un conjunto de 11 indicadores presentados en Power BI. Los resultados muestran una relación clara entre los retrasos y una menor satisfacción, y una relación más modesta con la recompra, lo que sustenta acciones preventivas en logística y atención al cliente.

---

2. Parte 1 — Las bases del modelo

2.1 Empresa o institución seleccionada

Olist (2018) es una empresa brasileña de comercio electrónico que se describe como integradora de marketplaces: conecta a pequeños negocios de todo el país con distintos canales de venta mediante un único contrato. Los comerciantes ofrecen sus productos en la tienda Olist Store y los despachan directamente a los clientes a través de socios logísticos de la empresa. En su descripción, Olist se presenta como la mayor tienda por departamentos dentro de los marketplaces de Brasil.

Los datos del proyecto provienen del conjunto público que la propia empresa publicó en Kaggle y que reúne información de alrededor de 100 000 pedidos realizados entre 2016 y 2018 en múltiples marketplaces de Brasil. El archivo de pedidos contiene 99 441 registros (Olist, 2018). Como los datos corresponden a ese periodo específico, los hallazgos describen el desempeño de la empresa en dicho lapso y no necesariamente su situación actual.

Para efectos de este proyecto, el equipo asume el rol de Última Milla Labs, un equipo de analítica de datos que apoya a Olist en la toma de decisiones. Se trata de un ejercicio académico: Olist no encargó este estudio.

Nombre	Olist
Tipo de negocio	Integrador de marketplaces de comercio electrónico (conecta pequeños comerciantes con canales de venta).
País	Brasil
Fuente de la información	Brazilian E-Commerce Public Dataset by Olist (Kaggle).
Periodo de los datos	Septiembre de 2016 a octubre de 2018.
Rol del equipo	Equipo de analítica de datos que apoya a la empresa.

2.2 Objetivos estratégicos de la empresa

En el material disponible para este proyecto no consta una declaración formal de los objetivos estratégicos de Olist. Por ello, los siguientes se plantean para efectos de este estudio y se derivan de su modelo de negocio: un marketplace que conecta a pequeños comerciantes con compradores y cuyo valor depende de que el pedido llegue completo, a tiempo y con una buena experiencia de compra (Olist, 2018).

N.º	**Objetivo estratégico**	**Cómo se mide**
OE1	Cumplir los plazos de entrega. Reducir la proporción de pedidos que llegan después de la fecha estimada, con atención a las regiones donde las entregas tardan más.	% de entregas tardías, retraso promedio y brecha de entrega entre estados (sección 4.2).
OE2	Elevar la satisfacción del cliente. Mejorar la experiencia de compra, medida con la calificación que los clientes otorgan al recibir su pedido.	Calificación promedio, % de reseñas negativas y brecha por puntualidad.
OE3	Fortalecer la fidelización. Favorecer que los clientes que compran por primera vez regresen a comprar en la plataforma.	Tasa de nueva compra y recompra según la experiencia del primer pedido.

2.3 Problema y necesidad del negocio

2.3.1 Descripción del problema

El problema central es que una parte de los pedidos llega después de la fecha estimada de entrega, que esas entregas tardías se asocian con una caída marcada en la satisfacción de los clientes y que se concentran en regiones determinadas del país, en un contexto donde pocos clientes regresan a comprar. Se considera entrega tardía aquella cuya fecha de entrega al cliente es posterior a la fecha estimada.

La exploración se hizo en PostgreSQL sobre los pedidos entregados con fecha de entrega entre enero de 2017 y agosto de 2018 (vista v_pedidos_base; 96 144 pedidos). Sus resultados son los siguientes:

- **Entregas.** El 8.07 % de los pedidos llegó tarde. Cuando el vendedor y el cliente están en el mismo estado, la entrega tardó en promedio 7.9 días; cuando están en estados distintos, 15.1 días.
- **Geografía.** El porcentaje de entregas tardías varía mucho según el estado del cliente: los más altos son Alagoas (23.8 %), Maranhão (19.6 %), Piauí (16.0 %) y Sergipe (15.4 %), y los más bajos São Paulo (5.9 %), Minas Gerais (5.6 %) y Paraná (5.0 %).
- **Satisfacción.** La calificación promedio general es de 4.16: 4.30 en los pedidos puntuales y 2.57 en los que llegaron tarde. Recibió una reseña de 1 o 2 estrellas el 9.2 % de los pedidos puntuales, frente al 53.9 % de los tardíos.
- **Fidelización.** Solo el 3.12 % de los clientes realizó más de un pedido. Volvió a comprar el 4.34 % de quienes recibieron a tiempo su primer pedido, frente al 2.94 % de quienes lo recibieron tarde.

Estos resultados evidencian asociaciones y no demuestran por sí mismos que el retraso cause la insatisfacción o la falta de recompra. Tampoco se dispone de información sobre las posibles causas de los retrasos, como el transporte o la ruta utilizada.

2.3.2 Necesidades del negocio

- Anticipar, al momento de la compra, qué pedidos tienen mayor riesgo de retraso.
- Identificar perfiles de pedidos y clientes según la experiencia que han tenido, para enfocar acciones.
- Contar con indicadores que permitan dar seguimiento al desempeño de la entrega, la satisfacción y la fidelización.

2.3.3 Hipótesis

Hipótesis central. Las entregas tardías se asocian con una menor satisfacción del cliente y con una menor probabilidad de que vuelva a comprar; además, parte del riesgo de retraso puede anticiparse con información disponible al momento de la compra, como la ubicación del cliente y del vendedor, el plazo estimado, el flete y las características del producto.

De esta hipótesis central se derivan las siguientes hipótesis específicas:

- **H1.** Los pedidos entregados después de la fecha estimada presentan calificaciones menores que los entregados a tiempo.
- **H2.** La proporción de entregas tardías varía según la región del cliente y según si el vendedor y el cliente se encuentran en el mismo estado.
- **H3.** Un modelo de clasificación entrenado con información disponible al momento de la compra identifica los pedidos con riesgo de retraso con un desempeño superior al de una línea base simple.
- **H4.** Existen grupos diferenciados de pedidos o clientes, según su valor, ubicación, forma de pago y experiencia de entrega, que presentan niveles distintos de satisfacción.

Las hipótesis se formulan en términos de asociación y no de causalidad. Surgen de una exploración preliminar de los datos y se contrastarán formalmente en el análisis (sección 4.5).

2.3.4 Objetivo general y objetivos específicos

Objetivo general. Desarrollar un modelo de análisis de datos para Olist que, mediante un modelo supervisado, uno no supervisado y un conjunto de indicadores, permita anticipar las entregas tardías y caracterizar perfiles de pedidos y clientes según su experiencia de compra, con el fin de apoyar los objetivos de cumplir los plazos de entrega, elevar la satisfacción de los clientes y fortalecer su fidelización.

Objetivos específicos:

- Integrar y depurar los datos fuente, documentando los problemas de calidad detectados y las decisiones de transformación aplicadas.
- Construir y validar dos modelos de aprendizaje automático: uno supervisado que prediga la entrega tardía con información disponible al comprar, y uno no supervisado que segmente pedidos o clientes en perfiles según valor, ubicación, pago y experiencia de entrega.

- Definir al menos 10 indicadores (KPI) alineados con los objetivos estratégicos y utilizarlos para analizar la asociación entre la entrega tardía, la satisfacción del cliente y una posible nueva compra, incluida su variación por regiones.
- Interpretar los resultados, formular recomendaciones y señalar cómo se deben actualizar y monitorear los modelos a lo largo del tiempo.

2.3.5 Metodología

El análisis se desarrolla como un flujo por etapas en KNIME: obtención, calidad, transformación, modelado, validación e indicadores. Cada etapa se describe en la sección 3 y los indicadores se presentan en Power BI (sección 4).

2.4 Datos fuente

Los datos provienen del conjunto público Brazilian E-Commerce Public Dataset by Olist, publicado por la propia empresa en Kaggle (Olist, 2018). Se descargaron el 4 de octubre de 2026 en formato CSV (codificación UTF-8), en nueve archivos que suman aproximadamente 127 MB y cubren pedidos realizados entre septiembre de 2016 y octubre de 2018.

Archivo	**Contenido**	**Filas**	**Columnas**
olist_orders_dataset	Pedidos, estado y fechas clave del proceso	99 441	8
olist_customers_dataset	Clientes, identificador único y ubicación	99 441	5
olist_order_items_dataset	Ítems de cada pedido: producto, vendedor, precio y flete	112 650	7
olist_order_payments_dataset	Pagos: tipo, cuotas y valor	103 886	5
olist_order_reviews_dataset	Calificación de 1 a 5 y comentarios	99 224	7
olist_products_dataset	Atributos de los productos	32 951	9
olist_sellers_dataset	Vendedores y su ubicación	3 095	4
olist_geolocation_dataset	Coordenadas por prefijo postal	1 000 163	5
product_category_name_translation	Traducción de categorías de producto	71	2

Ubicación y administración. Los datos son administrados y publicados por Olist en Kaggle. El equipo trabaja con una copia local de la descarga, que no se modifica. Los conjuntos derivados se generan como vistas en PostgreSQL (v_pedidos_base y v_clientes) y dentro del flujo de KNIME, sin modificar las tablas originales.

Ciclo de vida. El conjunto es una captura estática de un periodo cerrado y el equipo no lo actualizará. Dentro del proyecto, el ciclo consiste en: descarga única, almacenamiento de la copia original, transformación y análisis, conservación hasta la finalización del proyecto y posterior eliminación. Ninguna persona ajena al equipo interactúa con la información ni la observa.

Sensibilidad y seguridad. Los archivos no incluyen nombres, correos electrónicos, documentos de identificación ni direcciones exactas. Los identificadores de pedidos, clientes, vendedores y productos son códigos de 32 caracteres, y la ubicación se limita a un prefijo postal, ciudad o estado. No obstante, los comentarios de texto libre de las reseñas podrían contener información personal incidental. Una revisión preliminar mediante patrones automáticos no halló correos ni documentos; detectó dos coincidencias con formato de teléfono celular y dos enlaces entre

más de 40 000 comentarios, sin que esta revisión fuera exhaustiva. Como medida, el acceso a los datos se limita a los integrantes del equipo y en la presentación no se mostrarán registros individuales. Se respetan los términos de uso del conjunto: licencia CC BY-NC-SA 4.0 (https://creativecommons.org/licenses/by-nc-sa/4.0/). Los comentarios de texto no se utilizan en los modelos predictivos ni en la modelación analítica.

2.5 Infraestructura de base de datos

Se utilizó **PostgreSQL** ejecutándose localmente mediante Docker, en lugar de Snowflake. Esta decisión fue funcional y reproducible para todo el equipo: el esquema quedó versionado en el repositorio con el script `schema.sql`, la instancia se levantó de forma local y el flujo de ETL se conectó directamente con KNIME. La estructura final se implementó en el esquema `olist`, con la base de datos `postgres` configurada dentro del contenedor `olist-db`.

Se documentó la evidencia de funcionamiento mediante consultas SQL sobre el esquema y sobre la vista `v_pedidos_base`, así como la validación de conteos por tabla. La instancia quedó disponible para el desarrollo y la verificación del análisis de negocio.

2.6 Estructuras de datos creadas

El modelo de datos se compone de nueve estructuras (tablas) que corresponden a los archivos originales, creadas en el esquema olist de PostgreSQL mediante el script schema.sql (Anexo A y carpeta /datos del repositorio). Las decisiones de diseño fueron:

- Llaves primarias simples y compuestas según el diccionario de datos (order_items, order_payments y order_reviews usan llave compuesta).
- Llaves foráneas impuestas: orders → customers; order_items → orders, products y sellers; order_payments → orders; order_reviews → orders.
- Relaciones lógicas no impuestas: products → traducción de categorías (hay productos sin traducción) y los prefijos postales hacia geolocation (un prefijo se repite en muchas filas).
- Montos en NUMERIC(10,2) y fechas en TIMESTAMP. Se conservan los nombres originales de las columnas, incluidos los errores ortográficos de product_name_lenght y product_description_lenght, para facilitar la carga.
- Índices sobre las llaves foráneas y sobre customer_unique_id, que se usa para medir la recompra.

Fragmento representativo del script:

```
CREATE TABLE orders (
  order_id                      VARCHAR(32) PRIMARY KEY,
  customer_id                   VARCHAR(32) NOT NULL REFERENCES customers (customer_id),
  order_status                  VARCHAR(11) NOT NULL,
  order_purchase_timestamp      TIMESTAMP   NOT NULL,
  order_approved_at             TIMESTAMP,
  order_delivered_carrier_date  TIMESTAMP,
  order_delivered_customer_date TIMESTAMP,
  order_estimated_delivery_date TIMESTAMP   NOT NULL
);

CREATE TABLE order_items (
  order_id            VARCHAR(32)   NOT NULL REFERENCES orders (order_id),
```

```
  order_item_id       INTEGER       NOT NULL,
  product_id          VARCHAR(32)   NOT NULL REFERENCES products (product_id),
  seller_id           VARCHAR(32)   NOT NULL REFERENCES sellers (seller_id),
  shipping_limit_date TIMESTAMP     NOT NULL,
  price               NUMERIC(10,2) NOT NULL,
  freight_value       NUMERIC(10,2) NOT NULL,
  PRIMARY KEY (order_id, order_item_id)
);
```

2.7 Carga de los datos en las estructuras

Tabla	Filas en el CSV	Filas en PostgreSQL	¿Coincide?
orders	99 441	99 441	sí
customers	99 441	99 441	sí
order_items	112 650	112 650	sí
order_payments	103 886	103 886	sí
order_reviews	99 224	99 224	sí
products	32 951	32 951	sí
sellers	3 095	3 095	sí
geolocation	1 000 163	1 000 163	sí
product_category_name_translation	71	71	sí

La carga se realizó con el comando \copy de psql dentro del contenedor olist-db, respetando el orden de las llaves foráneas: traducción de categorías, clientes, vendedores, productos, pedidos, ítems, pagos, reseñas y geolocalización. Para verificarla se compara el conteo de filas de cada tabla con el del CSV:

```
SELECT 'orders' AS tabla, COUNT(*) AS filas FROM olist.orders
UNION ALL SELECT 'customers',      COUNT(*) FROM olist.customers
UNION ALL SELECT 'order_items',    COUNT(*) FROM olist.order_items
UNION ALL SELECT 'order_payments', COUNT(*) FROM olist.order_payments
UNION ALL SELECT 'order_reviews',  COUNT(*) FROM olist.order_reviews
UNION ALL SELECT 'products',       COUNT(*) FROM olist.products
UNION ALL SELECT 'sellers',        COUNT(*) FROM olist.sellers
UNION ALL SELECT 'geolocation',    COUNT(*) FROM olist.geolocation
UNION ALL SELECT 'translation',    COUNT(*) FROM olist.product_category_name_translation;
```

La verificación de carga quedó validada con la comparación entre filas del CSV y filas en PostgreSQL en todas las tablas del esquema.

2.8 Diccionario de datos

El diccionario indica, para cada estructura, su nombre y descripción; para cada campo, su nombre, tipo y descripción; y las relaciones entre estructuras. Los tipos se expresan en SQL estándar y se ajustan al motor elegido. PK indica llave primaria y FK llave foránea.

2.8.1 Estructuras

Estructura	Descripción	Llave primaria
orders	Pedidos realizados con estado y fechas clave del proceso	order_id
customers	Clientes asociados a cada pedido y su ubicación	customer_id
order_items	Ítems de cada pedido con producto, vendedor, precio y flete	order_id, order_item_id
order_payments	Pagos de cada pedido con medio, cuotas y valor	order_id, payment_sequential
order_reviews	Calificación y comentarios de la encuesta de satisfacción	review_id, order_id
products	Atributos de los productos	product_id
sellers	Vendedores y su ubicación	seller_id
geolocation	Coordenadas por prefijo postal	Ninguna natural
product_category_name_translation	Traducción de categorías de producto al inglés	product_category_name

2.8.2 Campos

Tabla: orders

Campo	Tipo	Llave	Nulos	Descripción
order_id	VARCHAR(32)	PK	No	Identificador único del pedido.
customer_id	VARCHAR(32)	FK	No	Identificador del cliente en ese pedido (se genera por pedido).
order_status	VARCHAR(11)		No	Estado del pedido (approved, canceled, created, delivered, invoiced, processing, shipped, unavailable).
order_purchase_timestamp	TIMESTAMP		No	Fecha y hora de la compra.
order_approved_at	TIMESTAMP		Sí	Fecha y hora de aprobación del pago.
order_delivered_carrier_date	TIMESTAMP		Sí	Fecha en que el pedido se entregó al socio logístico.
order_delivered_customer_date	TIMESTAMP		Sí	Fecha de entrega al cliente.
order_estimated_delivery_date	TIMESTAMP		No	Fecha estimada de entrega informada al cliente al comprar.

Tabla: customers

Campo	Tipo	Llave	Nulos	Descripción
customer_id	VARCHAR(32)	PK	No	Identificador del cliente asociado a un pedido.
customer_unique_id	VARCHAR(32)		No	Identificador único del cliente; se mantiene entre pedidos.
customer_zip_code_prefix	VARCHAR(5)		No	Prefijo del código postal (5 dígitos).
customer_city	VARCHAR(50)		No	Ciudad del cliente.
customer_state	CHAR(2)		No	Estado del cliente (sigla).

Tabla: order_items

Campo	Tipo	Llave	Nulos	Descripción
order_id	VARCHAR(32)	PK, FK	No	Pedido al que pertenece el ítem.
order_item_id	INTEGER	PK	No	Número secuencial del ítem dentro del pedido.
product_id	VARCHAR(32)	FK	No	Producto del ítem.
seller_id	VARCHAR(32)	FK	No	Vendedor del ítem.
shipping_limit_date	TIMESTAMP		No	Fecha límite para que el vendedor entregue el pedido al socio logístico.
price	NUMERIC(10,2)		No	Precio del ítem.
freight_value	NUMERIC(10,2)		No	Valor del flete asignado al ítem.

Tabla: order_payments

Campo	Tipo	Llave	Nulos	Descripción
order_id	VARCHAR(32)	PK, FK	No	Pedido pagado.
payment_sequential	INTEGER	PK	No	Secuencia de pago dentro del pedido (un pedido puede pagarse con varios medios).
payment_type	VARCHAR(20)		No	Medio de pago (boleto, credit_card, debit_card, voucher, not_defined).
payment_installments	INTEGER		No	Número de cuotas.
payment_value	NUMERIC(10,2)		No	Valor del pago.

Tabla: order_reviews

Campo	Tipo	Llave	Nulos	Descripción
review_id	VARCHAR(32)	PK	No	Identificador de la reseña; no es único por sí solo.
order_id	VARCHAR(32)	PK, FK	No	Pedido evaluado.
review_score	INTEGER		No	Calificación del 1 al 5.
review_comment_title	VARCHAR(50)		Sí	Título del comentario (opcional).
review_comment_message	VARCHAR(300)		Sí	Texto del comentario (opcional).
review_creation_date	TIMESTAMP		No	Fecha en que se envió la encuesta de satisfacción.
review_answer_timestamp	TIMESTAMP		No	Fecha y hora de la respuesta a la encuesta.

Tabla: products

Campo	Tipo	Llave	Nulos	Descripción
product_id	VARCHAR(32)	PK	No	Identificador único del producto.
product_category_name	VARCHAR(50)	FK lógica	Sí	Categoría del producto (en portugués).
product_name_lenght	INTEGER		Sí	Cantidad de caracteres del nombre (el campo conserva el error ortográfico original).

Campo	Tipo	Llave	Nulos	Descripción
product_description_lenght	INTEGER		Sí	Cantidad de caracteres de la descripción (conserva el error ortográfico original).
product_photos_qty	INTEGER		Sí	Cantidad de fotos del producto.
product_weight_g	INTEGER		Sí	Peso en gramos.
product_length_cm	INTEGER		Sí	Largo en centímetros.
product_height_cm	INTEGER		Sí	Alto en centímetros.
product_width_cm	INTEGER		Sí	Ancho en centímetros.

Tabla: sellers

Campo	Tipo	Llave	Nulos	Descripción
seller_id	VARCHAR(32)	PK	No	Identificador único del vendedor.
seller_zip_code_prefix	VARCHAR(5)		No	Prefijo del código postal (5 dígitos).
seller_city	VARCHAR(50)		No	Ciudad del vendedor.
seller_state	CHAR(2)		No	Estado del vendedor.

Tabla: geolocation

Campo	Tipo	Llave	Nulos	Descripción
geolocation_zip_code_prefix	VARCHAR(5)		No	Prefijo del código postal (5 dígitos).
geolocation_lat	DOUBLE PRECISION		No	Latitud.
geolocation_lng	DOUBLE PRECISION		No	Longitud.
geolocation_city	VARCHAR(50)		No	Ciudad.
geolocation_state	CHAR(2)		No	Estado (sigla).

Tabla: product_category_name_translation

Campo	Tipo	Llave	Nulos	Descripción
product_category_name	VARCHAR(50)	PK	No	Nombre de la categoría en portugués.
product_category_name_english	VARCHAR(50)		No	Nombre de la categoría en inglés.

Las columnas derivadas del flujo se documentan en la sección 3.3, donde se describen los indicadores y variables creadas para el análisis de entrega, satisfacción y recompras.

2.8.3 Relaciones entre estructuras

Origen	Destino	Cardinalidad	Observación
orders.customer_id	customers.customer_id	1 : 1	Cada pedido genera su propio customer_id.
order_items.order_id	orders.order_id	N : 1	Un pedido tiene de 1 a 21 ítems.

Origen	Destino	Cardinalidad	Observación
order_ items.product_id	products.product_id	N : 1	
order_items.seller_ id	sellers.seller_id	N : 1	
order_ payments.order_id	orders.order_id	N : 1	Un pedido tiene de 1 a 29 pagos.
order_ reviews.order_id	orders.order_id	N : 1	Algunos pedidos tienen más de una reseña y otros ninguna.
products.product_ category_name	product_category_name_ translation.product_category_ name	N : 1 (lógica)	Hay categorías sin traducción y productos sin categoría.
customers.customer_ zip_code_prefix	geolocation.geolocation_zip_ code_prefix	N : M (lógica)	El prefijo se repite en geolocation.
sellers.seller_zip_ code_prefix	geolocation.geolocation_zip_ code_prefix	N : M (lógica)	Mismo caso que la anterior.

2.9 Diagrama entidad-relación

Se incorpora el diagrama entidad-relación del modelo relacional usado en el proyecto. La versión fuente se conserva en la carpeta del repositorio y el esquema quedó alineado con la base implementada en PostgreSQL.

Enlace al diagrama: https://drive.google.com/file/d/1xBKpA89fh4JQ7KKysahj6yNuwq9vtR2d/view?usp=sharing

Cómo leer el diagrama:

- Línea continua: relación que puede imponerse como llave foránea.
- Línea punteada: relación lógica, de tres tipos: la traducción de categorías hacia products y las dos relaciones por código postal con geolocation (no puede imponerse porque existen prefijos sin coincidencia).
- Cardinalidades: orders y customers son 1 a 1, porque cada pedido genera su propio customer_id. orders se relaciona 1 a muchos, con posibles ceros, con ítems, pagos y reseñas, porque hay pedidos sin ítems, sin pago o sin reseña. products y sellers aparecen en muchos ítems.
- Llaves compuestas: order_items, order_payments y order_reviews combinan order_id con otro campo.

---

3. Parte 2 — Flujo y modelos de aprendizaje automático

3.1 Descripción general del flujo en KNIME

El análisis se desarrolla como un flujo por etapas en KNIME, desde la obtención de los datos hasta la validación de los modelos y el cálculo de los indicadores. La estructura del proyecto

refleja una separación clara entre ETL, clustering y predicción, con una capa final de indicadores para la toma de decisiones.

Etapa	**Qué se hace**	**Workflow group**
1. Obtención	Lectura de los nueve archivos con un nodo CSV Reader por archivo, desde la copia versionada en `knime/datos/` (ruta relativa al workflow). Si se usa la base de datos del proyecto, solo cambia el nodo de entrada (PostgreSQL Connector + DB Table Selector); el resto del flujo es igual.	01-ETL
2. Calidad	Perfilado de los datos (nulos, duplicados, rangos y consistencia de fechas) y aplicación de reglas (3.2).	01-ETL
3. Transformación	Integración de las tablas a nivel de pedido y creación de variables derivadas (3.3).	01-ETL
4. Modelado	Construcción de los dos modelos: no supervisado (3.4) y supervisado (3.5).	02-Clustering y 03-Prediccion
5. Validación	Ajuste de parámetros de cada modelo y documentación de los cambios (3.6).	02 y 03
6. Indicadores	Cálculo de los KPI sobre los datos transformados y presentación en Power BI (sección 4).	04-KPIs

3.2 Obtención y calidad de los datos

Se perfilaron los datos (nulos, duplicados, rangos y consistencia de fechas) y se aplicaron las siguientes reglas:

Problema o decisión	**Dónde**	**Acción tomada**	**Justificación**
Cobertura temporal desigual	orders	Se trabaja con pedidos entregados, con fecha de entrega entre enero de 2017 y agosto de 2018 (96 144 pedidos).	Baja cobertura de 2016 y de septiembre y octubre de 2018.
Más de una reseña por pedido	order_reviews	Se usa una calificación por pedido: el promedio cuando hay más de una.	Evitar duplicar pedidos al integrar tablas.
Códigos postales con ceros a la izquierda	customers, sellers, geolocation	Se leen como texto (String) y no como número.	Un prefijo como `01037` leído como entero pasa a `1037` y no coincidiría en los cruces por código postal.
Pedidos entregados sin fecha de entrega	orders	8 pedidos con `order_status = delivered` no tienen `order_delivered_customer_date`; la ventana de fechas de entrega los excluye.	No se puede medir la puntualidad sin fecha real de entrega.
Categoría de producto sin traducción	products, traducción	1 376 pedidos (1.43 %) quedan sin `product_category_name_english`; un nodo Missing Value los rellena con «desconocida».	Conservar los pedidos sin perder registros.

Problema o decisión	Dónde	Acción tomada	Justificación
Atributos de producto ausentes	products	Se tratan como categoría «desconocida».	Conservar los pedidos sin perder registros.
Valores atípicos e inconsistencias	Varias tablas	Se documentan, limitan o excluyen según el caso.	Mantener la calidad analítica sin perder la población del estudio.

La población de análisis se concentra en pedidos entregados en el periodo observado, con foco en la comparación entre puntualidad, satisfacción y recompra.

3.3 Transformación de los datos

Se integraron las tablas a nivel de pedido y se crearon las siguientes variables derivadas. Las categorías se traducen y agrupan, las variables categóricas se codifican y las numéricas se normalizan cuando se requiere.

Variable derivada	Cálculo	Uso
es_tardio	1 si order_delivered_customer_date >order_estimated_delivery_date; 0 en caso contrario	Variable objetivo del Modelo 2 y KPI de entregas tardías.
dias_retraso	order_delivered_customer_date − order_estimated_delivery_date (días; negativo si llegó antes)	Gravedad del retraso; variable del Modelo 1.
dias_entrega	order_delivered_customer_date − order_purchase_timestamp (días)	Tiempo de entrega; variable del Modelo 1.
plazo_estimado_ dias	order_estimated_delivery_date − order_purchase_timestamp (días)	Variable explicativa del Modelo 2.
mismo_estado	1 si customer_state = seller_state; 0 en caso contrario	Efecto de la distancia geográfica.
relacion_precio_ flete	freight_value ÷ price	Variable explicativa.
peso, volumen	product_weight_g; largo × alto × ancho	Variables explicativas.
num_items, cuotas, mes	Conteo de ítems; payment_ installments; mes de compra	Variables explicativas.
review_score_ pedido	Promedio de review_score por pedido	Satisfacción; no se usa como entrada de los modelos.
review_negativa	1 si review_score_pedido ≤ 2; 0 en caso contrario	KPI de reseñas negativas.
num_pedidos_ cliente, recompra	Conteo de pedidos por customer_ unique_id; 1 si es mayor que 1	KPI de fidelización.
primer_pedido_ tardio	es_tardio del primer pedido de cada cliente	KPI de recompra según puntualidad.

La vista base mantiene una fila por pedido, con agregación del valor total y del flete, y con una única referencia de vendedor y producto basada en el primer ítem del pedido. Esta decisión evita duplicar el pedido al integrar las tablas de items, pagos y reseñas.

Implementación en KNIME (workflow 01-ETL, 39 nodos). Los nueve CSV se leen con nodos CSV Reader, con los códigos postales como texto. Items, pagos y reseñas se agregan por

`order_id` con GroupBy (suma de precio y flete y máximo de `order_item_id` como número de ítems; suma de pagos y máximo de cuotas; promedio de calificación). Cuatro Joiner unen orders con customers y con esas tres agregaciones (Left Outer, para no perder pedidos) y un Row Filter conserva los pedidos `delivered`. Una rama aparte toma el primer ítem de cada pedido (`order_item_id = 1`) y lo une con sellers, products y la traducción de categorías para obtener `seller_state`, el peso, las medidas y la categoría; esa rama se une al flujo principal por `order_id`.

Las variables derivadas se calculan así: un Rule Engine crea `mismo_estado` (1 si `customer_state = seller_state`); String to Date&Time convierte las fechas conservando la hora; un Row Filter aplica la ventana de entrega (enero de 2017 a agosto de 2018); tres nodos Date&Time Difference miden en segundos el plazo estimado, el tiempo de entrega y el retraso, y tres Math Formula los pasan a días con decimales (`plazo_estimado_dias`, `dias_entrega`, `dias_retraso`); un Rule Engine crea `es_tardio` (1 si el retraso en segundos es mayor que 0) y dos Math Formula calculan `relacion_precio_flete` y `volumen_cm3`. Se mide en segundos porque los Date&Time Difference solo permiten decimales con granularidad de tiempo, no con días, y así un pedido entregado el día estimado, pero después de las 00:00, cuenta como tardío. Un Column Filter deja 29 columnas, un Missing Value rellena la categoría ausente con «desconocida», un nodo Statistics resume la tabla final, un nodo Date&Time to String deja las fechas con formato fijo `yyyy-MM-dd HH:mm:ss` y un CSV Writer la guarda en `knime/datos/ordenes_etl.csv` para los workflows 02 y 03.

Columna del ETL en KNIME	**Equivale a**
total_precio, total_flete	valor_items, flete_total
num_items	num_items
total_pagado, max_cuotas	valor_pagado, cuotas
review_promedio	review_score_pedido
dias_entrega, dias_retraso, plazo_estimado_dias	dias_entrega, dias_retraso, plazo_estimado_dias
es_tardio, mismo_estado, relacion_precio_flete	es_tardio, mismo_estado, relacion_precio_flete
volumen_cm3, product_weight_g	volumen_cm3, peso_g
product_category_name_english	categoria

El peso, el volumen y la categoría se toman del primer ítem del pedido, igual que en v_pedidos_base. Quedan fuera de la tabla final las columnas intermedias (identificadores del ítem, segundos, ciudad del vendedor y atributos del producto que no se usan).

3.4 Modelo 1 — No supervisado: segmentación con k-means

El modelo no supervisado agrupa los pedidos según su experiencia de compra y entrega, con k-means sobre seis variables: valor del pedido (`total_precio`), costo del flete (`total_flete`), número de cuotas (`max_cuotas`), tiempo de entrega (`dias_entrega`), retraso respecto de la fecha estimada (`dias_retraso`) y si el cliente y el vendedor están en el mismo estado (`mismo_estado`). La calificación no se usa como entrada, para que la comparación de satisfacción entre grupos (H4) no sea circular. El precio y el flete se transforman con logaritmo, porque tienen colas muy largas, y las seis variables se normalizan con puntuación z para que todas pesen lo mismo en la distancia. Ninguna de las seis tiene valores ausentes. La lista de nodos está en el Anexo D.

Origen de las cifras. El perfil de los segmentos (3.4.2), la prueba de H4 y la silueta de k = 4 salen del workflow de KNIME (`02-Clustering`, k = 4, semilla 2; la asignación está en `knime/datos/segmentos_modelo1.csv`). La tabla de k de 2 a 8 (3.4.1) y el análisis de sensibilidad se calcularon en Python (scikit-learn) con los mismos datos y variables. Las diferencias pequeñas

con KNIME, por ejemplo 7 357 frente a 7 423 pedidos en el grupo problemático, se deben a la inicialización aleatoria del k-means.

3.4.1 Resultados y validación

Se probaron valores de k entre 2 y 8 y se compararon con dos criterios: la inercia (método del codo) y el coeficiente de silueta, calculado sobre una muestra aleatoria de 10 000 pedidos.

k	**Inercia**	**Silueta**
2	444 062	0.272
3	368 642	0.287
4	**301 593**	**0.305**
5	273 977	0.272
6	253 415	0.231
7	240 525	0.230
8	222 060	0.221

Se eligió $\mathbf{k = 4}$: tiene la silueta más alta y coincide con el codo, porque de 4 a 5 grupos la inercia baja mucho menos que de 3 a 4. El agrupamiento es estable: con cinco semillas distintas se obtiene prácticamente la misma asignación (índice de Rand ajustado de 0.999 o más). Una silueta de 0.30 indica una estructura moderada: los grupos existen, pero sus fronteras no son nítidas, por lo que se usan para describir y priorizar, no como categorías rígidas. En KNIME, con el nodo Silhouette Coefficient sobre las primeras 10 000 filas de la tabla, la silueta global de $k = 4$ es 0.297, casi igual que la de referencia (0.305); por grupo va de 0.11 a 0.36, así que hay un grupo que se separa con claridad y otros que se solapan más.

Análisis de sensibilidad. Como `dias_entrega` y `dias_retraso` solo se conocen después de la entrega, se repitió el modelo con las cuatro variables disponibles al momento de la compra (precio, flete, cuotas y mismo estado). En ese caso el mejor valor es $k = 3$ (silueta 0.391), pero las calificaciones de los grupos apenas varían, entre 4.00 y 4.29. Es decir, la segmentación comercial casi no distingue la satisfacción; lo que la distingue es la experiencia de entrega.

3.4.2 Perfil e interpretación de los segmentos

Segmento	**Pedidos**	**Precio (mediana, BRL)**	**Cuotas (prom.)**	**Entrega (días)**	**Mismo estado**	**Entregas tardías**	**Calificación**	**Reseñas negativas**
Compras grandes a plazos	16 767 (17.4 %)	219.00	7.1	12.4	22 %	2.6 %	4.16	12.7 %
Estándar entre estados	41 511 (43.2 %)	69.90	1.9	12.4	0 %	0.7 %	4.31	8.6 %
Locales rápidas	30 509 (31.7 %)	64.99	2.1	7.3	99 %	4.5 %	4.32	8.9 %
Entregas problemáticas	7 357 (7.7 %)	98.00	2.8	35.1	9 %	77.4 %	2.59	52.8 %

- **Compras grandes a plazos:** pedidos de mayor valor pagados en muchas cuotas; casi siempre llegan a tiempo, pero su calificación es algo menor que la de los pedidos baratos, lo que sugiere expectativas más altas.

- **Estándar entre estados:** el grupo más grande; pedidos baratos entre estados distintos que, pese a tardar unos 12 días, casi nunca llegan después de la fecha estimada.
- **Locales rápidas:** cliente y vendedor en el mismo estado; la entrega más rápida y la mejor calificación.
- **Entregas problemáticas:** 7.7 % de los pedidos, con 35 días de entrega en promedio, 77 % de entregas tardías y más de la mitad de reseñas negativas. Es el segmento donde se concentra la mala experiencia.

Este último segmento coincide en gran parte con los pedidos tardíos, porque el retraso es una de las variables de entrada. Por eso el modelo debe leerse como una segmentación de la experiencia de entrega: confirma y dimensiona el patrón de H1, más que descubrir un factor nuevo.

3.5 Modelo 2 — Supervisado: predicción de la entrega tardía

El modelo supervisado clasifica si un pedido llegará tarde o a tiempo usando únicamente variables conocidas al momento de la compra. Este enfoque evita filtrado de información futura y permite evaluar la utilidad del modelo para actuar de forma preventiva antes de que se produzca la entrega.

Se comparan dos algoritmos, regresión logística y Random Forest, con dos líneas base: predecir siempre «a tiempo» y una regla geográfica que marca como tardío todo pedido entre estados distintos. La mejora se mide con indicadores orientados a la clase minoritaria: AUC, precisión, exhaustividad y F1 de la entrega tardía. La guía de nodos está en `documento/guia-modelo2-knime.md`.

Variables de entrada (13): `total_precio`, `total_flete`, `num_items`, `max_cuotas`, `relacion_precio_flete`, `product_weight_g`, `volumen_cm3`, `plazo_estimado_dias`, `mismo_estado`, `customer_state`, `seller_state`, `product_category_name_english` y el día de la semana de la compra. Se excluyen todas las variables que solo se conocen después de la compra (fechas de aprobación, despacho y entrega, días de entrega, retraso y calificación). También se excluyó el mes de compra: con solo 20 meses de datos, el modelo aprendía los picos de retraso de un único año (en las pruebas era la variable más importante del Random Forest), un patrón que no se puede asumir que se repita.

3.5.1 Datos de entrenamiento y de prueba

La partición se define en forma temporal para evitar fuga de información: el conjunto de entrenamiento cubre el período inicial y la prueba cubre el período más reciente. La clase tardía se mantiene minoritaria, por lo que se prioriza la evaluación basada en sensibilidad y AUC sobre la exactitud.

Con un corte en el 1 de abril de 2018 sobre la fecha de compra, el entrenamiento reúne 64 052 pedidos con 9.19 % de entregas tardías y la prueba 32 092 con 5.85 %. La tasa de la prueba es menor porque el porcentaje de retrasos varió mucho entre meses: noviembre de 2017 (14.31 %), febrero de 2018 (15.97 %) y marzo de 2018 (21.36 %) fueron meses atípicos y quedan en el entrenamiento, mientras que desde abril de 2018 baja a niveles cercanos al 5 % al 10 %. Por eso el AUC y las líneas base se evalúan sobre una clase tardía menos frecuente que en el entrenamiento, y las métricas de la prueba no son directamente comparables con las del entrenamiento. La partición no es inválida, pero esta diferencia se tiene en cuenta al interpretar los resultados.

3.5.2 Preparación, parámetros e hiperparámetros

Todo lo que se aprende de los datos se calcula solo con el entrenamiento y se aplica después a la prueba: la imputación de los 16 valores ausentes de peso y volumen (mediana), la normalización de las variables numéricas para la regresión logística (puntuación z) y el balanceo de clases (muestreo de igual tamaño de tardíos y puntuales, solo en entrenamiento). En los resultados de referencia, el Random Forest usó 200 árboles y un tamaño mínimo de hoja de 20, y el desbalance se trató con ponderación de clases. [Pendiente: elegir el tamaño mínimo de hoja con validación cruzada de 5 particiones estratificadas dentro del entrenamiento (comparar 10, 20 y 50) y anotar aquí el valor elegido y su AUC promedio.] La prueba temporal se usa una sola vez, al final.

3.6 Validación de ambos modelos

Origen de las cifras. Las cifras de los modelos de esta sección se calcularon en Python (scikit-learn) con los mismos datos, variables y partición.

Modelo 2 sobre la prueba (abril a agosto de 2018; 32 092 pedidos, 1 878 tardíos; umbral 0.5):

Modelo	**AUC**	**Precisión**	**Exhaustividad**	**F1**
Línea base 1: siempre a tiempo	0.500	0.000	0.000	0.000
Línea base 2: estados distintos	0.472	0.053	0.545	0.097
Random Forest	0.625	0.078	0.444	0.132
Regresión logística	**0.721**	**0.100**	**0.782**	**0.177**

La regresión logística supera a ambas líneas base y al Random Forest, y cumple el criterio propuesto para H3 (AUC mayor que 0.60 y mejor que las líneas base). Que un modelo lineal gane a uno de árboles es poco común, pero ocurre cuando la señal es débil y casi lineal, como aquí. Las variables más importantes del Random Forest fueron el plazo estimado, el costo del flete, el estado del cliente (Río de Janeiro y São Paulo), si cliente y vendedor están en el mismo estado y el precio.

Análisis de errores. La regresión logística detecta 78 de cada 100 pedidos tardíos, pero de cada 10 pedidos que marca como tardíos solo 1 lo es (1 469 aciertos y 13 084 falsas alarmas). Un falso negativo deja pasar un pedido de riesgo y un falso positivo genera una alerta innecesaria; con esta precisión el modelo sirve para priorizar el seguimiento de pedidos, no para tomar decisiones automáticas sobre cada uno. Si se cambia el umbral, debe elegirse con la validación cruzada del entrenamiento y nunca mirando la prueba.

La regla geográfica falla en la prueba. En el entrenamiento, los pedidos entre estados distintos llegaron tarde mucho más que los del mismo estado (11.0 % frente a 5.7 %); en la prueba la relación se invierte (5.3 % frente a 6.6 %). Por eso la línea base 2 queda por debajo del azar (AUC 0.47). La relación entre geografía y retraso no es estable en el tiempo (ver H2 en la sección 4.5).

Modelo 1. Su validación interna (silueta, codo y estabilidad entre semillas) está en la sección 3.4.1, y su validación externa —si los grupos difieren en satisfacción— es la prueba de H4 (sección 4.5).

3.7 Integraciones con terceros

No se incorporaron integraciones con terceros. El análisis se basa en variables estructuradas de la base de datos y en los modelos ejecutados dentro del flujo de KNIME, sin dependencia de servicios externos para la parte analítica principal.

3.8 Evidencia de ejecución del flujo

La evidencia del flujo requiere mostrar que la cadena completa corre de manera coherente desde la lectura de datos hasta la validación de resultados. En el proyecto, la prueba de ejecución se apoya en la propia validación relacional de tablas, las vistas analíticas y los resultados del cálculo de KPI, que son consistentes con la población de análisis y la lógica de negocio definida.

Ejecución del workflow 01-ETL. El flujo se ejecutó completo en KNIME con sus 39 nodos. Tras los cuatro Joiner la tabla tiene 99 441 filas (una por pedido, igual que orders, sin duplicados). Después del filtro de pedidos entregados y de la ventana de fechas de entrega quedan 96 144 filas, que coincide con el conteo de v_pedidos_base, y `order_id` no se repite. El promedio de `es_tardio` es 0.0807 (8.07 %), igual al KPI 1. En la tabla final solo faltan valores en `review_promedio` (642 pedidos sin reseña), en peso y medidas (16 pedidos), en `order_approved_at` (14) y en `order_delivered_carrier_date` (1). El nodo Statistics advierte que no calcula frecuencias de las columnas con más de 1 000 valores distintos, como los identificadores y las fechas, lo que no afecta a las columnas analíticas. [Pendiente: capturas de pantalla del flujo ejecutado.]

---

4. Parte 3 — Visualización e indicadores

4.1 Conexión entre KNIME y Power BI

Los resultados del flujo se exportan desde KNIME y se consumen en Power BI como tablas analíticas de pedidos y clientes. La conexión se apoya en los indicadores ya calculados en PostgreSQL y en las vistas creadas para la capa analítica, permitiendo mantener una estructura consistente entre la parte técnica y la visualización.

4.2 Indicadores (KPI)

Se definen once indicadores, organizados según el objetivo estratégico al que responden. Los valores reportados se apoyan en la base validada de PostgreSQL y en las vistas analíticas del proyecto.

OE1 — Cumplir los plazos de entrega

N.º	KPI	Definición	Relevancia	Valor
1	Porcentaje de entregas tardías	Pedidos entregados después de la fecha estimada ÷ total de pedidos entregados	Mide directamente el incumplimiento de los plazos prometidos.	8.07 %
2	Tiempo promedio de entrega	Promedio de días entre la compra y la entrega	Indica la velocidad general del servicio logístico.	12.5 días

N.º	KPI	Definición	Relevancia	Valor
3	Retraso promedio de las entregas tardías	Promedio de días entre la fecha estimada y la fecha real, solo en pedidos tardíos	Mide la gravedad del incumplimiento y no solo su frecuencia.	9.4 días
4	Brecha de entrega entre estados	Tiempo promedio de entrega cuando cliente y vendedor están en estados distintos, comparado con cuando están en el mismo	Identifica el efecto de la distancia geográfica sobre el plazo.	15.1 días frente a 7.9 días
5	Tasa de pedidos no concretados	Pedidos cancelados o no disponibles ÷ total de pedidos	Refleja los pedidos que no llegaron a entregarse.	1.24 %

OE2 — Elevar la satisfacción del cliente

N.º	KPI	Definición	Relevancia	Valor
6	Calificación promedio	Promedio de la calificación de la reseña	Resume la satisfacción general.	4.16
7	Porcentaje de reseñas negativas	Pedidos con calificación de 1 o 2 ÷ pedidos con reseña	Capta la proporción de clientes insatisfechos.	12.73 %
8	Brecha de calificación por puntualidad	Calificación promedio de los pedidos puntuales menos la de los tardíos	Cuantifica cuánto se asocia el retraso con una menor satisfacción.	1.73 puntos (4.30 frente a 2.57)

OE3 — Fortalecer la fidelización

N.º	KPI	Definición	Relevancia	Valor
9	Tasa de nueva compra	Clientes únicos con más de un pedido ÷ total de clientes únicos	Mide la fidelización.	3.12 %
10	Recompra según la puntualidad del primer pedido	% de clientes con un segundo pedido, entre quienes recibieron su primer pedido a tiempo y quienes lo recibieron con retraso	Evalúa si la experiencia de entrega se asocia con una compra futura. Se calcula solo con clientes cuyo primer pedido fue en 2017 y se entregó en el periodo (42 076 clientes), para dar tiempo a la recompra.	4.34 % (a tiempo) frente a 2.94 % (tardío)
11	Recompra según la calificación del primer pedido	Igual que el anterior, comparando clientes con reseña negativa y no negativa en su primer pedido (41 761 clientes con reseña, primer pedido en 2017)	Evalúa si la satisfacción se asocia con una compra futura.	3.71 % (negativa) frente a 4.32 % (no negativa)

4.3 Dashboard

La estructura del dashboard contempla una vista de resumen, otra de entregas, otra de satisfacción y otra de fidelización, con filtros por período y por estado del cliente en todas las

páginas.

Página	KPI	Visuales
1. Resumen	1, 2, 6, 9	Tarjetas con los cuatro KPI y línea mensual del % de entregas tardías y de la calificación promedio.
2. Entregas (OE1)	1 a 5	Mapa o barras por estado; barras de tiempo de entrega en el mismo estado vs. estados distintos; histograma de días de retraso.
3. Satisfacción (OE2)	6 a 8	Calificación promedio de pedidos puntuales vs. tardíos; % de reseñas negativas por tramo de retraso (a tiempo, 1 a 3, 4 a 7 y más de 7 días).
4. Fidelización (OE3)	9 a 11	Tasa de recompra según la puntualidad y según la calificación del primer pedido.
5. Perfiles y riesgo	—	Tabla de segmentos del Modelo 1 (tamaño, valor y calificación promedio); distribución de la probabilidad de retraso y cantidad de pedidos en riesgo alto del Modelo 2.

Las medidas DAX de los KPI están en el Anexo C y las tablas de origen (v_pedidos_base y v_clientes) en el Anexo A.

4.4 Hallazgos principales

N.º	Hallazgo	Evidencia	Acción recomendada
1	Las entregas tardías se asocian con una caída fuerte de la satisfacción: 4.30 frente a 2.57 puntos.	KPI 8; reseñas negativas 9.2 % vs. 53.9 %.	Contactar de forma proactiva a los clientes con pedidos tardíos (aviso, compensación).
2	El retraso se concentra en pocos estados: 23.8 % de entregas tardías en Alagoas y 19.6 % en Maranhão frente a 5.9 % en São Paulo.	KPI 1 por estado.	Revisar socios logísticos y plazos prometidos en los estados críticos.
3	Entre estados distintos la entrega tarda casi el doble: 15.1 frente a 7.9 días.	KPI 4.	Ajustar el plazo mostrado al cliente en rutas entre estados; fomentar vendedores cercanos.
4	La recompra es baja (3.12 %) y la diferencia según la puntualidad del primer pedido es pequeña (4.34 % vs. 2.94 %); según la reseña es aún menor (4.32 % vs. 3.71 %).	KPI 9 y 10.	Verificar si la diferencia es significativa antes de atribuirla al retraso; explorar otros factores.
5	El 7.6 % de los pedidos forma un segmento de «entregas problemáticas» (35 días de entrega, 78 % tardías, calificación 2.58); la segmentación comercial, sin variables de entrega, casi no distingue la satisfacción.	Modelo 1 (sección 3.4).	Enfocar la atención al cliente en la experiencia logística más que en el tipo de compra.

N.º	Hallazgo	Evidencia	Acción recomendada
6	Con información disponible al comprar, la regresión logística anticipa el retraso mejor que las líneas base (AUC 0.72), pero con baja precisión. El plazo estimado y el flete son las señales más fuertes.	Modelo 2 (sección 3.6).	Usar la probabilidad de retraso para priorizar el seguimiento, no para decisiones automáticas.
7	La relación entre geografía y retraso cambió en 2018: entre abril y agosto, los pedidos del mismo estado llegaron tarde más que los de estados distintos.	Sección 3.6; partición temporal.	Monitorear los KPI por región cada mes y reentrenar el modelo de forma periódica.

4.5 Contraste de las hipótesis

Hipótesis	Qué se compara	Prueba propuesta	Resultado y conclusión
H1	Calificación de pedidos puntuales vs. tardíos	Mann-Whitney U; chi-cuadrado para % de reseñas negativas	Se confirma. Calificación 4.30 frente a 2.57 (Mann-Whitney U = 149 338 900, p <0.001; correlación biserial de rangos 0.55, efecto grande). Reseñas negativas 9.2 % frente a 53.9 % (² = 12 589, gl = 1, p <0.001; V de Cramér 0.36).
H2	% de entregas tardías por región y por mismo estado / estados distintos	Chi-cuadrado de independencia	Se confirma para el periodo completo, con un matiz temporal. Por estado del cliente: ² = 1 621, gl = 26, p <0.001 (V = 0.13). Mismo estado frente a distinto: 6.0 % frente a 9.2 % de tardíos (² = 301, gl = 1, p <0.001; V = 0.06, efecto pequeño). La dirección no es estable: hasta marzo de 2018, 5.7 % frente a 11.0 %; de abril a agosto de 2018 se invierte, 6.6 % frente a 5.3 % (² = 22.8, p <0.001).
H3	Desempeño del modelo vs. línea base	AUC, precisión, exhaustividad y F1 de la clase tardía	Se confirma. La regresión logística obtiene AUC 0.72 en la prueba temporal, frente a 0.50 y 0.47 de las líneas base, y supera el umbral de 0.60. Su precisión es baja (0.10). [Confirmar con la salida de KNIME.]

Hipótesis	Qué se compara	Prueba propuesta	Resultado y conclusión
H4	Calificación promedio entre los grupos del Modelo 1	Kruskal-Wallis o ANOVA	Se confirma, con una advertencia. Las calificaciones de los cuatro grupos van de 2.58 a 4.32 (Kruskal-Wallis H = 7 789, p <0.001). La diferencia se debe sobre todo al grupo de entregas problemáticas, que se forma con las variables de retraso; con solo las variables de compra la diferencia es pequeña (de 4.00 a 4.29; H = 507). En KNIME, el One-way ANOVA sobre los 95 502 pedidos con reseña da F = 4 477.7 (3 y 95 498 grados de libertad, p <0.001) y un tamaño de efecto de 0.12 (suma de cuadrados entre grupos 19 358 sobre un total de 156 979). El grupo de entregas problemáticas (7 357 pedidos, 77 % tardíos) promedia 2.60 y los otros tres entre 4.16 y 4.32. Como la calificación es ordinal, Kruskal-Wallis es la prueba más adecuada, y ambas concluyen lo mismo.

Los valores de las pruebas se calcularon en Python sobre `knime/datos/ordenes_etl.csv`, excluyendo los 642 pedidos sin reseña en las pruebas de calificación.

Recompra (KPI 10 y 11). La diferencia de recompra según la puntualidad del primer pedido (4.34 % frente a 2.94 %) es estadísticamente significativa ($^2 \approx 12.3$, p <0.001), pero pequeña: 1.4 puntos porcentuales. Según la reseña del primer pedido (4.32 % frente a 3.71 %), apenas alcanza significancia ($^2 \approx 3.9$, p ≈ 0.047). Ambas pruebas se calcularon a partir de los conteos y porcentajes de `datos/resultados_kpis.txt`, por lo que el valor es aproximado.

Dos cuidados al interpretar. (1) Con muestras grandes casi cualquier diferencia resulta estadísticamente significativa; además del valor p hay que reportar el tamaño de la diferencia (puntos porcentuales o puntos de calificación) y si importa para el negocio. (2) Las entregas tardías se concentran en ciertas regiones; para H1 y para los KPI 10 y 11 conviene comparar también dentro de cada región, para no atribuir al retraso lo que podría deberse a la región.

5. Conclusiones y recomendaciones

- **Hipótesis central y H1.** Las entregas tardías se asocian con una caída de 1.73 puntos en la calificación promedio (4.30 frente a 2.57) y con una proporción mucho mayor de reseñas negativas (9.2 % frente a 53.9 %). La evidencia sugiere que la puntualidad no es solo un indicador operativo, sino un factor asociado con la percepción del servicio.
- **H2.** El retraso se concentra en regiones específicas y, en el periodo completo, es mayor cuando cliente y vendedor están en estados distintos. Sin embargo, esa relación no fue

estable: entre abril y agosto de 2018 se invirtió. La localización importa, pero su efecto cambia con el tiempo, por lo que las decisiones logísticas deben basarse en un monitoreo continuo y no en una regla fija.

- **Fidelización.** La recompra es baja (3.12 %). La diferencia por puntualidad del primer pedido es estadísticamente significativa, pero pequeña (1.4 puntos porcentuales: 4.34 % frente a 2.94 %), por lo que la fidelización requiere un enfoque más amplio que solo la entrega a tiempo.
- **Modelos (H3 y H4).** La regresión logística anticipa el retraso al momento de la compra mejor que las líneas base (AUC 0.72), aunque con baja precisión, por lo que sirve para priorizar seguimiento y no para decisiones automáticas. La segmentación identifica un 7.6 % de pedidos con muy mala experiencia de entrega; la satisfacción depende sobre todo de la logística, no del tipo de compra. Ninguno de los dos modelos establece causalidad.

Recomendaciones para Olist:

- Priorizar la mejora logística en los estados con mayor proporción de entregas tardías y revisar el plazo estimado que se muestra al cliente en rutas entre estados distintos.
- Usar el modelo de riesgo para identificar, al momento de la compra, los pedidos con alta probabilidad de retraso y actuar de forma proactiva (aviso al cliente, ajuste del plazo prometido o escalamiento con el socio logístico).
- Dar seguimiento mensual a los KPI del dashboard y reentrenar los modelos de forma periódica, con revisión de la estabilidad regional y del comportamiento de recompra.

6. Limitaciones y trabajo futuro

- Los resultados describen asociaciones y no causalidad.
- No se dispone de información sobre las causas de los retrasos (transporte, ruta, vendedor o temporada).
- Los datos corresponden a un periodo cerrado (2016–2018); los hallazgos pueden no reflejar la situación actual de Olist.
- Solo alrededor del 3 % de los clientes recompra y alrededor del 8 % de los pedidos llega tarde: las clases están desbalanceadas, lo que exige métricas y umbrales distintos de la exactitud.
- Solo una parte de los clientes deja reseña, por lo que la satisfacción medida puede estar sesgada.
- La confusión regional puede exagerar la asociación entre retraso y satisfacción, porque ciertos estados concentran tanto el retraso como las condiciones de compra.
- La recompra se observa con un horizonte temporal limitado, y algunos clientes aún no han tenido suficiente tiempo para volver a comprar.
- El plazo estimado lo fija la empresa con sus propios criterios; es informativo para el modelo, pero no necesariamente es la causa del retraso.
- Trabajo futuro: actualización y monitoreo de los modelos, nuevas variables, análisis del texto de los comentarios (en portugués) y prueba de otros algoritmos.

7. Referencias y anexos

Olist. (2018). Brazilian E-Commerce Public Dataset by Olist [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

Documentación de KNIME, PostgreSQL y Power BI consultada durante la construcción del flujo y la visualización, en formato APA y conforme a la bibliografía del curso.

Anexos

Anexo A. Scripts SQL: estructuras y vistas analíticas

A.1 schema.sql

```
-- schema.sql: estructuras del proyecto MO-0005 (PostgreSQL)
-- Orden de carga: translation, customers, sellers, products, orders,
-- order_items, order_payments, order_reviews, geolocation.

CREATE SCHEMA IF NOT EXISTS olist;
SET search_path TO olist;

CREATE TABLE product_category_name_translation (
  product_category_name         VARCHAR(50) PRIMARY KEY,
  product_category_name_english VARCHAR(50) NOT NULL
);

CREATE TABLE customers (
  customer_id              VARCHAR(32) PRIMARY KEY,
  customer_unique_id       VARCHAR(32) NOT NULL,
  customer_zip_code_prefix VARCHAR(5)  NOT NULL,
  customer_city            VARCHAR(50) NOT NULL,
  customer_state           CHAR(2)     NOT NULL
);

CREATE TABLE sellers (
  seller_id              VARCHAR(32) PRIMARY KEY,
  seller_zip_code_prefix VARCHAR(5)  NOT NULL,
  seller_city            VARCHAR(50) NOT NULL,
  seller_state           CHAR(2)     NOT NULL
);

CREATE TABLE products (
  product_id                 VARCHAR(32) PRIMARY KEY,
  product_category_name      VARCHAR(50),  -- relacion logica (no FK)
  product_name_lenght        INTEGER,      -- se conserva el nombre original
  product_description_lenght INTEGER,      -- se conserva el nombre original
  product_photos_qty         INTEGER,
  product_weight_g           INTEGER,
  product_length_cm          INTEGER,
  product_height_cm          INTEGER,
  product_width_cm           INTEGER
);

CREATE TABLE orders (
  order_id                      VARCHAR(32) PRIMARY KEY,
  customer_id                   VARCHAR(32) NOT NULL REFERENCES customers (customer_id),
  order_status                  VARCHAR(11) NOT NULL,
```

```
  order_purchase_timestamp      TIMESTAMP   NOT NULL,
  order_approved_at             TIMESTAMP,
  order_delivered_carrier_date  TIMESTAMP,
  order_delivered_customer_date TIMESTAMP,
  order_estimated_delivery_date TIMESTAMP   NOT NULL
);

CREATE TABLE order_items (
  order_id            VARCHAR(32)   NOT NULL REFERENCES orders (order_id),
  order_item_id       INTEGER       NOT NULL,
  product_id          VARCHAR(32)   NOT NULL REFERENCES products (product_id),
  seller_id           VARCHAR(32)   NOT NULL REFERENCES sellers (seller_id),
  shipping_limit_date TIMESTAMP     NOT NULL,
  price               NUMERIC(10,2) NOT NULL,
  freight_value       NUMERIC(10,2) NOT NULL,
  PRIMARY KEY (order_id, order_item_id)
);

CREATE TABLE order_payments (
  order_id             VARCHAR(32)   NOT NULL REFERENCES orders (order_id),
  payment_sequential   INTEGER       NOT NULL,
  payment_type         VARCHAR(20)   NOT NULL,
  payment_installments INTEGER       NOT NULL,
  payment_value        NUMERIC(10,2) NOT NULL,
  PRIMARY KEY (order_id, payment_sequential)
);

CREATE TABLE order_reviews (
  review_id               VARCHAR(32)  NOT NULL,
  order_id                VARCHAR(32)  NOT NULL REFERENCES orders (order_id),
  review_score            INTEGER      NOT NULL,
  review_comment_title    VARCHAR(50),
  review_comment_message  VARCHAR(300),
  review_creation_date    TIMESTAMP    NOT NULL,
  review_answer_timestamp TIMESTAMP    NOT NULL,
  PRIMARY KEY (review_id, order_id)
);

CREATE TABLE geolocation (
  geolocation_zip_code_prefix VARCHAR(5)       NOT NULL,
  geolocation_lat             DOUBLE PRECISION NOT NULL,
  geolocation_lng             DOUBLE PRECISION NOT NULL,
  geolocation_city            VARCHAR(50)      NOT NULL,
  geolocation_state           CHAR(2)          NOT NULL
);

CREATE INDEX idx_orders_customer   ON orders (customer_id);
CREATE INDEX idx_items_product     ON order_items (product_id);
CREATE INDEX idx_items_seller      ON order_items (seller_id);
CREATE INDEX idx_reviews_order     ON order_reviews (order_id);
CREATE INDEX idx_customers_unique  ON customers (customer_unique_id);
CREATE INDEX idx_geo_zip           ON geolocation (geolocation_zip_code_prefix);
```

A.2 views.sql (poblaciones base: pedidos entregados entre enero de 2017 y agosto de 2018, y un registro por cliente)

```
-- views.sql: dataset analitico a nivel de pedido y de cliente

SET search_path TO olist;

-- Poblacion base: pedidos entregados con fecha de entrega
-- entre enero de 2017 y agosto de 2018.
-- dias_retraso es negativo si el pedido llego antes de lo estimado.
```

```
CREATE OR REPLACE VIEW v_pedidos_base AS
WITH items AS (
  SELECT order_id,
         COUNT(*)           AS num_items,
         SUM(price)         AS valor_items,
         SUM(freight_value) AS flete_total
  FROM order_items GROUP BY order_id
), primer_item AS (   -- vendedor y producto de referencia: primer item
  SELECT order_id, product_id, seller_id
  FROM order_items WHERE order_item_id = 1
), pago AS (
  SELECT order_id,
         MAX(payment_installments) AS cuotas,
         SUM(payment_value)        AS valor_pagado
  FROM order_payments GROUP BY order_id
), resena AS (        -- una calificacion por pedido (promedio)
  SELECT order_id, ROUND(AVG(review_score), 2) AS review_score_pedido
  FROM order_reviews GROUP BY order_id
)
SELECT o.order_id,
       c.customer_unique_id,
       c.customer_state,
       s.seller_state,
       o.order_purchase_timestamp,
       o.order_delivered_customer_date,
       o.order_estimated_delivery_date,
       EXTRACT(EPOCH FROM (o.order_delivered_customer_date - o.order_purchase_timestamp)) /
    86400.0 AS dias_entrega,
       EXTRACT(EPOCH FROM (o.order_estimated_delivery_date - o.order_purchase_timestamp)) /
    86400.0 AS plazo_estimado_dias,
       EXTRACT(EPOCH FROM (o.order_delivered_customer_date - o.order_estimated_delivery_date))
     / 86400.0 AS dias_retraso,
       CAST(o.order_delivered_customer_date > o.order_estimated_delivery_date AS INTEGER) AS
    es_tardio,
       CAST(c.customer_state = s.seller_state AS INTEGER) AS mismo_estado,
       i.num_items,
       i.valor_items,
       i.flete_total,
       i.flete_total / NULLIF(i.valor_items, 0) AS relacion_precio_flete,
       pg.cuotas,
       pg.valor_pagado,
       p.product_weight_g AS peso_g,
       p.product_length_cm * p.product_height_cm * p.product_width_cm AS volumen_cm3,
       COALESCE(t.product_category_name_english, 'desconocida') AS categoria,
       r.review_score_pedido
FROM orders o
JOIN customers c        ON c.customer_id = o.customer_id
LEFT JOIN items i       ON i.order_id = o.order_id
LEFT JOIN primer_item f ON f.order_id = o.order_id
LEFT JOIN sellers s     ON s.seller_id = f.seller_id
LEFT JOIN products p    ON p.product_id = f.product_id
LEFT JOIN product_category_name_translation t ON t.product_category_name = p.
    product_category_name
LEFT JOIN pago pg       ON pg.order_id = o.order_id
LEFT JOIN resena r      ON r.order_id = o.order_id
WHERE o.order_status = 'delivered'
  AND o.order_delivered_customer_date >= '2017-01-01'
  AND o.order_delivered_customer_date <  '2018-09-01';

-- Un registro por cliente: caracteristicas de su PRIMER pedido y
-- cantidad total de pedidos (todos los estados).
CREATE OR REPLACE VIEW v_clientes AS
WITH pedidos AS (
```

```
  SELECT c.customer_unique_id,
         o.order_id,
         o.order_purchase_timestamp,
         b.es_tardio,
         b.review_score_pedido,
         ROW_NUMBER() OVER (PARTITION BY c.customer_unique_id
                            ORDER BY o.order_purchase_timestamp, o.order_id) AS n_pedido,
         COUNT(*) OVER (PARTITION BY c.customer_unique_id) AS total_pedidos
  FROM orders o
  JOIN customers c ON c.customer_id = o.customer_id
  LEFT JOIN v_pedidos_base b ON b.order_id = o.order_id
)
SELECT customer_unique_id,
       total_pedidos,
       EXTRACT(YEAR FROM order_purchase_timestamp) AS anio_primer_pedido,
       es_tardio AS primer_pedido_tardio,
       CASE WHEN review_score_pedido IS NULL THEN NULL
            WHEN review_score_pedido <= 2 THEN 1 ELSE 0 END AS primer_pedido_negativo
FROM pedidos
WHERE n_pedido = 1;
```

Anexo B. Consultas de los indicadores

Cada consulta indica la población que usa. Los KPI 1 a 4 y 6 a 8 se calculan sobre los pedidos entregados de la vista v_pedidos_base; el KPI 5, sobre todos los pedidos; los KPI 9 a 11, sobre clientes únicos.

```
-- kpis.sql: valores de los 11 KPI (poblacion indicada en cada caso)

SET search_path TO olist;

-- KPI 1. % de entregas tardias (v_pedidos_base)
SELECT ROUND(100.0 * AVG(es_tardio), 2) AS pct_tardias FROM v_pedidos_base;

-- KPI 2. Tiempo promedio de entrega (dias)
SELECT ROUND(AVG(dias_entrega), 1) AS dias_entrega_prom FROM v_pedidos_base;

-- KPI 3. Retraso promedio de las entregas tardias (dias)
SELECT ROUND(AVG(dias_retraso), 1) AS retraso_prom
FROM v_pedidos_base WHERE es_tardio = 1;

-- KPI 4. Brecha de entrega entre estados
SELECT mismo_estado,
       COUNT(*) AS pedidos,
       ROUND(AVG(dias_entrega), 1) AS dias_entrega_prom
FROM v_pedidos_base GROUP BY mismo_estado ORDER BY mismo_estado;

-- KPI 5. Tasa de pedidos no concretados (todos los pedidos)
SELECT ROUND(100.0 * AVG(CASE WHEN order_status IN ('canceled', 'unavailable')
                              THEN 1 ELSE 0 END), 2) AS pct_no_concretados
FROM orders;

-- KPI 6. Calificacion promedio
SELECT ROUND(AVG(review_score_pedido), 2) AS calificacion_prom
FROM v_pedidos_base WHERE review_score_pedido IS NOT NULL;

-- KPI 7. % de resenas negativas (1 o 2)
SELECT ROUND(100.0 * AVG(CASE WHEN review_score_pedido <= 2 THEN 1 ELSE 0 END), 2) AS
    pct_negativas
FROM v_pedidos_base WHERE review_score_pedido IS NOT NULL;
```

```sql
-- KPI 8. Brecha de calificacion por puntualidad
SELECT es_tardio,
       COUNT(*) AS pedidos,
       ROUND(AVG(review_score_pedido), 2) AS calificacion_prom,
       ROUND(100.0 * AVG(CASE WHEN review_score_pedido <= 2 THEN 1 ELSE 0 END), 1) AS
    pct_negativas
FROM v_pedidos_base WHERE review_score_pedido IS NOT NULL
GROUP BY es_tardio ORDER BY es_tardio;

-- KPI 9. Tasa de nueva compra (todos los clientes unicos)
SELECT COUNT(*) AS clientes,
       ROUND(100.0 * AVG(CASE WHEN total_pedidos > 1 THEN 1 ELSE 0 END), 2) AS pct_recompra
FROM v_clientes;

-- KPI 10. Recompra segun puntualidad del primer pedido (primer pedido en 2017)
SELECT primer_pedido_tardio,
       COUNT(*) AS clientes,
       ROUND(100.0 * AVG(CASE WHEN total_pedidos > 1 THEN 1 ELSE 0 END), 2) AS pct_recompra
FROM v_clientes
WHERE anio_primer_pedido = 2017 AND primer_pedido_tardio IS NOT NULL
GROUP BY primer_pedido_tardio ORDER BY primer_pedido_tardio;

-- KPI 11. Recompra segun la calificacion del primer pedido (primer pedido en 2017)
SELECT primer_pedido_negativo,
       COUNT(*) AS clientes,
       ROUND(100.0 * AVG(CASE WHEN total_pedidos > 1 THEN 1 ELSE 0 END), 2) AS pct_recompra
FROM v_clientes
WHERE anio_primer_pedido = 2017 AND primer_pedido_negativo IS NOT NULL
GROUP BY primer_pedido_negativo ORDER BY primer_pedido_negativo;

-- Exploracion: % de tardias por estado del cliente
SELECT customer_state, COUNT(*) AS pedidos,
       ROUND(100.0 * AVG(es_tardio), 1) AS pct_tardias
FROM v_pedidos_base GROUP BY customer_state ORDER BY pct_tardias DESC;

-- Exploracion: % de resenas negativas segun dias de retraso
SELECT CASE WHEN es_tardio = 0 THEN 'a tiempo'
            WHEN CEIL(dias_retraso) <= 3 THEN '1 a 3 dias'
            WHEN CEIL(dias_retraso) <= 7 THEN '4 a 7 dias'
            ELSE 'mas de 7 dias' END AS retraso,
       COUNT(*) AS pedidos,
       ROUND(100.0 * AVG(CASE WHEN review_score_pedido <= 2 THEN 1 ELSE 0 END), 1) AS
    pct_negativas
FROM v_pedidos_base WHERE review_score_pedido IS NOT NULL
GROUP BY 1 ORDER BY 1;
```

Anexo C. Medidas DAX para Power BI

```
% Entregas Tardías =
DIVIDE(CALCULATE(COUNTROWS(Pedidos), Pedidos[es_tardio] = 1), COUNTROWS(Pedidos))

Tiempo Promedio Entrega = AVERAGE(Pedidos[dias_entrega])

Retraso Promedio Tardías =
CALCULATE(AVERAGE(Pedidos[dias_retraso]), Pedidos[es_tardio] = 1)

Entrega Mismo Estado =
CALCULATE(AVERAGE(Pedidos[dias_entrega]), Pedidos[mismo_estado] = 1)

Entrega Estados Distintos =
CALCULATE(AVERAGE(Pedidos[dias_entrega]), Pedidos[mismo_estado] = 0)
```

```
Brecha Entrega Estados = [Entrega Estados Distintos] - [Entrega Mismo Estado]

% No Concretados =
DIVIDE(
  CALCULATE(COUNTROWS(PedidosTodos),
            PedidosTodos[order_status] IN {"canceled", "unavailable"}),
  COUNTROWS(PedidosTodos))

Calificación Promedio = AVERAGE(Pedidos[review_score_pedido])

% Reseñas Negativas =
DIVIDE(
  CALCULATE(COUNTROWS(Pedidos), Pedidos[review_score_pedido] <= 2),
  COUNT(Pedidos[review_score_pedido]))

Calificación Puntuales =
CALCULATE([Calificación Promedio], Pedidos[es_tardio] = 0)

Calificación Tardíos =
CALCULATE([Calificación Promedio], Pedidos[es_tardio] = 1)

Brecha Calificación = [Calificación Puntuales] - [Calificación Tardíos]

Tasa Nueva Compra =
DIVIDE(CALCULATE(COUNTROWS(Clientes), Clientes[total_pedidos] > 1),
       COUNTROWS(Clientes))

Recompra Primer Pedido Puntual =
CALCULATE([Tasa Nueva Compra],
          Clientes[anio_primer_pedido] = 2017, Clientes[primer_pedido_tardio] = 0)

Recompra Primer Pedido Tardío =
CALCULATE([Tasa Nueva Compra],
          Clientes[anio_primer_pedido] = 2017, Clientes[primer_pedido_tardio] = 1)

Recompra Primer Pedido Negativo =
CALCULATE([Tasa Nueva Compra],
          Clientes[anio_primer_pedido] = 2017, Clientes[primer_pedido_negativo] = 1)

Recompra Primer Pedido No Negativo =
CALCULATE([Tasa Nueva Compra],
          Clientes[anio_primer_pedido] = 2017, Clientes[primer_pedido_negativo] = 0)
```

Anexo D. Nodos de KNIME por etapa

Etapa	Nodos principales	Notas
Obtención	PostgreSQL Connector → DB Table Selector → DB Reader (uno por tabla)	Una conexión compartida; leer las nueve tablas. Sin base de datos: un CSV Reader por archivo con ruta `../datos/<archivo>.csv` (carpeta `knime/datos/`); es lo usado en la versión base del 01-ETL, con los códigos postales como String.
Calidad	Statistics, Missing Value, Row Filter, Rule Engine	Se documenta cada filtro con su conteo de filas antes y después (99 441, 96 478 y 96 144 pedidos en el ETL).

Etapa	Nodos principales	Notas
Transformación	Joiner, GroupBy, Date&Time Difference, Math Formula, Rule Engine, String Manipulation	Reproduce la vista v_pedidos_base (Anexo A); verificar que los conteos coincidan.
Preparación de modelos	Column Filter, Normalizer, One to Many (codificación), Row Splitter por fecha	Partición temporal: entrenamiento hasta marzo de 2018, prueba de abril a agosto de 2018.
Modelo 1	CSV Reader, Column Filter, Math Formula (logaritmo), Normalizer (Z-score), k-Means, Joiner, GroupBy, Rule-based Row Filter, Silhouette Coefficient, One-way ANOVA, CSV Writer	$k = 4$ con semilla 2; la silueta de $k = 4$ en KNIME es 0.297 y la comparación de k de 2 a 8 se calculó en Python. La prueba de H4 usa One-way ANOVA, porque el nodo de Kruskal-Wallis no está instalado. Detalle en `documento/guia-modelo1-knime.md`.
Modelo 2	Row Splitter (corte 2018-04-01), Missing Value y Missing Value (Apply), Equal Size Sampling, Normalizer y Normalizer (Apply), Random Forest Learner y Predictor, Logistic Regression Learner y Predictor, Rule Engine (líneas base), Scorer, Binary Classification Inspector	Balanceo solo sobre entrenamiento. Detalle en `documento/guia-modelo2-knime.md`.
Validación	Scorer, ROC Curve, Cross Validation (X-Partitioner / X-Aggregator)	Validación cruzada solo dentro del conjunto de entrenamiento.
Salida a Power BI	CSV Writer	Escribe `ordenes_etl.csv` y `segmentos_modelo1.csv` con el identificador del pedido.

Anexo E. Glosario

Término	Definición
Entrega tardía	Pedido cuya fecha de entrega al cliente es posterior a la fecha estimada.
Reseña negativa	Calificación de 1 o 2 en la escala de 1 a 5.
Recompra o nueva compra	Cliente único (customer_unique_id) con más de un pedido.
KPI	Indicador clave de desempeño: medida que permite dar seguimiento a un objetivo del negocio.
Línea base	Referencia simple contra la cual se compara el modelo para saber si aporta una mejora real.
AUC	Área bajo la curva ROC: mide qué tan bien el modelo separa pedidos tardíos de puntuales (0.5 equivale al azar; 1, a una separación perfecta).
Precisión	De los pedidos que el modelo marca como tardíos, la proporción que realmente lo es.
Exhaustividad (recall)	De los pedidos que realmente fueron tardíos, la proporción que el modelo logra detectar.
F1	Media armónica de la precisión y la exhaustividad.
K-means	Algoritmo no supervisado que agrupa las observaciones en k grupos según su cercanía.
Coeficiente de silueta	Medida entre -1 y 1 de qué tan bien separados y compactos están los grupos.

Término	Definición
Partición temporal	Entrenar con datos más antiguos y probar con datos más recientes, como ocurriría en la práctica.

Anexo F. Enlaces

- Repositorio del proyecto: https://github.com/FerretoDev/mo0005-ecommerce-ml
- Video del flujo en ejecución: PENDIENTE (agregar el enlace cuando esté grabado).
- Diagrama entidad-relación: https://drive.google.com/file/d/1xBKpA89fh4JQ7KKysahj6yNuwq9vtR2d/view?usp=sharing